Contents

Supplementary Material

Manuscript: "Variance-Reduced Trajectory Unravelings for GPU Noisy Quantum-Circuit Simulation: Characterization and a Qiskit-Aer Integration Gap" **Author:** Chun-Yeol You, Department of Physics and Chemistry, DGIST, Daegu 42988, Republic of Korea

Repository: https://github.com/mirryou-maker/qiskit-variance-reduced-unraveling.

This document provides the reproducibility package: exact environment, code map, run instructions, raw-data dictionary, and the derivations supporting the closed-form variance expressions.

S1. Environment manifest

All measurements reported in the manuscript were produced on the following single-GPU system. Any re-run must record its own manifest; numbers are hardware- and version-dependent for wall-clock, but the trajectory-count results are hardware-independent.

Component	Version / model
GPU	NVIDIA GeForce RTX 5060 Ti, 8 GB (Blackwell, sm_120)
GPU driver	596.49
Host OS	Windows 11 + WSL2, Ubuntu 24.04
Python	3.12.3 (venv `~/qsim-venv`)
qiskit	1.2.4
qiskit-aer-gpu	0.15.1
cupy	14.1.1 (cupy-cuda12x)
matplotlib	3.11.1
Random seed	20260719 (all experiments; per-block offsets noted in scripts)

Note on version pinning. The prebuilt `qiskit-aer-gpu` wheel is pinned at 0.15.1, which is incompatible with qiskit 2.x; qiskit is therefore pinned to 1.2.4. On the Blackwell GPU the wheel's kernels are JIT-compiled from PTX on first use (a one-time ~3 s cost), after which execution is native.

Reproducing the environment

```
wsl -d Ubuntu-24.04
python3 -m venv ~/qsim-venv && source ~/qsim-venv/bin/activate
pip install "qiskit>=1.1,<1.3" "qiskit-aer-gpu==0.15.1" cupy-cuda12x numpy matplotlib
python -c "from qiskit_aer import AerSimulator; print(AerSimulator().available_devices())"
# expected: ('CPU', 'GPU')
```

S2. Code map

Path	Role
`src/trajectory/gpu_trajectory.py`	GPU dense-statevector trajectory engine; gate contraction, `apply_projector`, `apply_analog_kick`, `expval_1q`, `prob_plus`
`validation/test_a1_tracedistance.py`	Correctness: ensemble ρ vs. Aer `density_matrix`; $1/\sqrt{N}$ convergence (Fig. 2)
`validation/test_a2_variance.py`	Closed-form variance reproduction (standard / projector / analog)
`validation/test_a3_sweep.py`	Regime map, n-scaling, unbiasedness sweep (Figs. 3–4)
`validation/test_c_caveats.py`	GHZ stabilizer observable; coherent-erosion map (Fig. 5)
`validation/test_a4_vs_aer.py`	Head-to-head vs. Aer `batched_shots_gpu` (Fig. 6)
`validation/test_a4b_aer_projector_kraus.py`	Attempt to inject projector unraveling through Aer's API
`validation/probe_aer_canonicalization.py`	Probe showing Aer stores/canonicalizes the channel
`paper/figures/make_figures.py`	Regenerates all six figures from the raw CSVs

Run order

```
source ~/qsim-venv/bin/activate
python -u validation/test_a1_tracedistance.py      # correctness
python -u validation/test_a2_variance.py           # closed-form variance
python -u validation/test_a3_sweep.py              # regime map + scaling (longest)
python -u validation/test_c_caveats.py             # GHZ + erosion
python -u validation/test_a4_vs_aer.py             # head-to-head
python -u validation/test_a4b_aer_projector_kraus.py cpu
python -u validation/test_a4b_aer_projector_kraus.py gpu
python -u validation/probe_aer_canonicalization.py
python -u paper/figures/make_figures.py            # figures
```

S3. Figure and table index

Item	Content	Produced by
Fig. 1	Concept: where this work intervenes in the Qiskit-Aer stack	`paper/figures/make_figures.py` (schematic)
Fig. 2	Correctness: trace distance vs. N, $1/\sqrt{N}$ convergence	`validation/test_a1_tracedistance.py`
Fig. 3	Regime map: variance ratio vs. noise strength	`validation/test_a3_sweep.py`
Fig. 4	Scaling of the trajectory-reduction factor vs. n	`validation/test_a3_sweep.py`
Fig. 5	Applicability map: coherent erosion of the advantage	`validation/test_c_caveats.py`
Fig. 6	Head-to-head vs. Qiskit-Aer	`validation/test_a4_vs_aer.py`
Table I	Differentiation from the closest related work	(narrative)
Table II	Regime map, numeric	`validation/test_a3_sweep.py`
Table III	Trajectories-to-target vs. n	`validation/test_a3_sweep.py`
Table IV	Head-to-head at n = 10	`validation/test_a4_vs_aer.py`

All figures are regenerated from the archived CSVs by `paper/figures/make_figures.py`; no figure contains data that is not present in `bench/results/`.

S4. Raw data dictionary

All raw outputs are archived as CSV under `bench/results/`, each with a comment header recording the environment, circuit, noise strength, and seed.

File	Contents	Figure/Table
`a1_convergence.csv`	trace distance vs. N; $D \cdot \sqrt{N}$	Fig. 2
`a2_variance.csv`	empirical vs. closed-form variance (3 unravelings, γt sweep); GPU N-to-target	—
`a3_sweep.csv`	regime map; circuit-class map; n-scaling; unbiasedness	Fig. 3, Fig. 4, Tables II–III
`a4_vs_aer.csv`	Aer vs. ours (mean, std, N-to-target, wall-clock)	Fig. 6, Table IV
`c_caveats.csv`	GHZ stabilizer speedups; erosion vs. θ	Fig. 5

S5. Derivations

S5.1 Projector unraveling reproduces the dephasing generator

With $\Pi_\pm = (\mathbb{1} \pm Z)/2$ and $L_\pm = \sqrt{2\gamma}\,\Pi_\pm$,

$$\sum_\pm L_\pm \rho L_\pm^\dagger = \tfrac{\gamma}{2}\left[(\mathbb{1}+Z)\rho(\mathbb{1}+Z) + (\mathbb{1}-Z)\rho(\mathbb{1}-Z)\right] = \gamma\left(\rho + Z\rho Z\right),$$

$$\sum_\pm L_\pm^\dagger L_\pm = 2\gamma\left(\Pi_+ + \Pi_-\right) = 2\gamma\,\mathbb{1} \implies \tfrac{1}{2}\{\textstyle\sum_\pm L_\pm^\dagger L_\pm, \rho\} = 2\gamma\rho.$$

Hence the dissipator is $\gamma(\rho + Z\rho Z) - 2\gamma\rho = \gamma(Z\rho Z - \rho)$, identical to the standard unraveling $L = \sqrt{\gamma}Z$. The two unravelings are therefore statistically distinguishable but physically equivalent.

S5.2 Estimator variances

For $O = X$, initial state $|+\rangle$, pure dephasing, no coherent evolution, the exact mean is $\langle X\rangle_t = e^{-2\gamma t}$.

Standard. Each jump applies Z, mapping $|+\rangle \leftrightarrow |-\rangle$, so $\langle X\rangle = (-1)^{N(t)}$ with $N(t) \sim \text{Poisson}(\gamma t)$. Then $\mathbb{E}[(-1)^N] = e^{-2\gamma t}$ and $\mathbb{E}[\langle X\rangle^2] = 1$, giving

$$\text{Var}_{\text{std}} = 1 - e^{-4\gamma t}.$$

Projector. Total jump rate is 2γ. The first jump projects onto a Z-eigenstate where $\{X, Z\} = 0$ forces $\langle X\rangle = 0$ permanently (absorbing window). The estimator is Bernoulli with success probability $e^{-2\gamma t}$ and values $\{1, 0\}$, so $\mathbb{E}[\langle X\rangle^2] = \mathbb{E}[\langle X\rangle] = e^{-2\gamma t}$ and

$$\text{Var}_{\text{proj}} = e^{-2\gamma t}\left(1 - e^{-2\gamma t}\right).$$

The ratio $\text{Var}_{\text{proj}}/\text{Var}_{\text{std}} = e^{-2\gamma t}/(1 + e^{-2\gamma t})$ tends to 0 as $\gamma t \to \infty$.

S5.3 GHZ stabilizer observable

For the GHZ state the single-qubit $\langle X_i\rangle$ vanishes identically. The stabilizer $X^{\otimes n}$ measures the $|0\rangle^{\otimes n}$–$|1\rangle^{\otimes n}$ coherence, decays as $e^{-2n\gamma t}$ under per-qubit dephasing, and anticommutes with every Z_i, so the absorbing-window argument applies with mean $m = e^{-2n\gamma t}$ and $\text{Var}_{\text{proj}} = m(1-m)$. It is evaluated on the statevector as $\langle X^{\otimes n}\rangle = \text{Re}\sum_x \psi_x^* \psi_{\bar{x}}$ with $\bar{x} = x \oplus (2^n - 1)$.

S6. Supporting evidence for the Qiskit-Aer canonicalization finding

The dephasing channel $\mathcal{E}(\rho) = (1-p)\rho + pZ\rho Z$ was supplied to Aer in two mathematically identical Kraus forms:

- standard: $\{\sqrt{1-p}\,\mathbb{1},\ \sqrt{p}\,Z\}$
- projector: $\{\sqrt{1-2p}\,\mathbb{1},\ \sqrt{2p}\,\Pi_+,\ \sqrt{2p}\,\Pi_-\}$

Both reproduce the correct mean, but both also yield the *same* (standard) estimator variance, i.e. N-to-target ≈ 1001 on CPU and ≈ 1011 in a per-trajectory expectation loop. Inspection (`probe_aer_canonicalization.py`) shows each error is stored as a single `kraus` instruction, and at apply time Aer reconstructs the Choi-canonical Kraus set — for dephasing, the orthogonal Pauli set — discarding the supplied decomposition.

That Aer's collapse machinery nonetheless exists is confirmed with amplitude damping, a non-unital channel whose canonical Kraus set is itself collapse-type: per-trajectory $\langle Z \rangle$ is bimodal (values $\{0.429, 1.0\}$; 29.6 % of trajectories collapsed). The proposed minimal change is therefore to (i) let a `QuantumError` carry a *preserve-unraveling* flag, (ii) sample K_i by $\|K_i\psi\|^2$ over the supplied set, and (iii) expose a per-trajectory expectation accumulator.

S7. Known limitations of the artifact

- The engine is a Python/cupy prototype; wall-clock comparisons against Aer's compiled C++/CUDA engine are reported as measured and are not the paper's efficiency claim (trajectory count is).
- `test_a3_sweep.py` is the longest run (tens of minutes at $n = 20$); reduce `N` or the n grid for a quick check.
- Results are for Pauli-Lindblad noise, where jump rates are state-independent and events can be sampled exactly. Non-Pauli channels require norm-dependent sampling and are not exercised here.